Assignment: Working with HTML and JSON

This is a warm up exercise to help you to get more familiar with the HTML and JSON file formats, and using packages to read these data formats for downstream use in R data frames. In the next two class weeks, we'll be loading these file formats from the web, using web scraping and web APIs.

Select **three books** on a single subject of your choice. At least **one book must have multiple authors**. For each book, record the **title**, **authors**, and **two to three additional attributes** of your choosing (e.g., publication year, publisher, ISBN, genre, rating).

Using this information, **manually create two data files**:

- An **HTML file** containing a table with the book data (e.g., `books.html`)
- A **JSON file** representing the same data (e.g., `books.json`)

Unless you are already very comfortable with these formats, you are expected to create both files **by hand** to better understand their structure.

Next, write **R code** (using packages of your choice) to:

1. Load the HTML data into an R data frame
2. Load the JSON data into a separate R data frame
3. Compare the two data frames and determine whether they are identical

Deliverables

- `books.html`
- `books.json`
- R code that loads and compares both data sources

Package your complete solution in a **Quarto (`.qmd`) file** and **publish it to rpubs.com**.

Note: Publishing will require making your HTML and JSON files **publicly accessible on the web**.

explain the content

Thought for a few seconds

This assignment is asking you to do a small practice project with **HTML, JSON, and R**.

What you need to do

Choose **3 books about the same subject**.

Rules:

- all 3 books should be on **one topic**
- **at least 1 book must have multiple authors**
- for each book, record:
  - **title**
  - **author(s)**
  - **2–3 extra attributes** such as publication year, publisher, ISBN, genre, or rating

Then create 2 data files by hand

You need to manually make:

1. `books.html`
   - an HTML file
   - it should contain a **table** with the book information
2. `books.json`
   - a JSON file
   - it should contain the **same book data** as the HTML file

The point is to practice understanding how HTML and JSON are structured, so they want you to **type both files yourself**, not auto-generate them.

After that, write R code to do 3 things

Your R code should:

1. **Load the HTML data** into an R data frame
2. **Load the JSON data** into another R data frame
3. **Compare the two data frames** and check whether they are identical

What you must submit

You need these items:

- `books.html`
- `books.json`
- R code that reads both files and compares them

Final format

Put the whole assignment into a **Quarto (`.qmd`) file** and publish it to **RPubs**.

Important note

Because you are publishing online, your HTML and JSON files need to be **publicly accessible on the web** so your R code can read them when the assignment is viewed.

In simple terms

You are basically doing this:

- pick 3 related books
- build the same dataset in **HTML** and **JSON**
- use **R** to read both

- prove they match
- submit everything in a **Quarto report**

If you want, I can next break this into:

1. what files you should create first, and
2. the exact structure for `books.html`, `books.json`, and the R code.

show me an example of html

Here is a simple example of an HTML file for this assignment.

html

```html
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Books Table</title>
</head>
<body>
  <h1>Books on Data Science</h1>

  <table border="1">
    <tr>
      <th>Title</th>
      <th>Authors</th>
      <th>Publication Year</th>
      <th>Publisher</th>
      <th>ISBN</th>
    </tr>
    <tr>
      <td>R for Data Science</td>
      <td>Hadley Wickham, Garrett Grolemund</td>
      <td>2017</td>
      <td>O'Reilly Media</td>
      <td>9781491910399</td>
```

```
    </tr>
    <tr>
      <td>Data Science for Business</td>
      <td>Foster Provost, Tom Fawcett</td>
      <td>2013</td>
      <td>O'Reilly Media</td>
      <td>9781449361327</td>
    </tr>
    <tr>
      <td>Practical Statistics for Data Scientists</td>
      <td>Peter Bruce, Andrew Bruce, Peter Gedeck</td>
      <td>2020</td>
      <td>O'Reilly Media</td>
      <td>9781492072942</td>
    </tr>
  </table>
</body>
</html>
```

What this does

- `<!DOCTYPE html>` : tells the browser this is an HTML5 file
- `<html>` : wraps the whole page
- `<head>` : contains page information like title and character encoding
- `<body>` : contains what shows on the page
- `<table>` : creates the table
- `<tr>` : table row
- `<th>` : header cell
- `<td>` : regular data cell

Copy code

For your assignment

You would save this as something like:

```html
books.html
```

Copy code

Then later R can read the table from this file.

Here is an even more minimal version:

html

```html
<html>
<body>
  <table border="1">
    <tr>
      <th>Title</th>
      <th>Authors</th>
      <th>Year</th>
    </tr>
    <tr>
      <td>Book A</td>
      <td>Author 1, Author 2</td>
      <td>2020</td>
    </tr>
  </table>
</body>
</html>
```

Next I can show you a matching `books.json` example for the same dat Copy code

explain each steps

Thought for a couple of seconds

Sure. Here is the same HTML example, with **each part explained step by step**.

Full example

html

```html
<!DOCTYPE html>
<html>
<head>
```

```
    <meta charset="UTF-8">
    <title>Books Table</title>
  </head>
  <body>
    <h1>Books on Data Science</h1>

    <table border="1">
      <tr>
        <th>Title</th>
        <th>Authors</th>
        <th>Publication Year</th>
        <th>Publisher</th>
        <th>ISBN</th>
      </tr>
      <tr>
        <td>R for Data Science</td>
        <td>Hadley Wickham, Garrett Grolemund</td>
        <td>2017</td>
        <td>O'Reilly Media</td>
        <td>9781491910399</td>
      </tr>
      <tr>
        <td>Data Science for Business</td>
        <td>Foster Provost, Tom Fawcett</td>
        <td>2013</td>
        <td>O'Reilly Media</td>
        <td>9781449361327</td>
      </tr>
      <tr>
        <td>Practical Statistics for Data Scientists</td>
        <td>Peter Bruce, Andrew Bruce, Peter Gedeck</td>
        <td>2020</td>
        <td>O'Reilly Media</td>
        <td>9781492072942</td>
      </tr>
    </table>
  </body>
  </html>
```

Step 1: `<!DOCTYPE html>`

Copy code

html

```
<!DOCTYPE html>
```

Copy code

This tells the browser:

- this file is an **HTML5** document
- read and display it using normal modern HTML rules

You usually put this at the **very top**.

Step 2: `<html>`

html

```
<html>
```

Copy code

This is the root element of the whole HTML page.

It means:

- everything inside is part of the HTML document

At the end, you close it with:

html

```
</html>
```

Copy code

Step 3: `<head>`

html

```
<head>
```

Copy code

The `<head>` section stores **page information**, not the main visible content.

Inside it, you usually put things like:

- page title
- character encoding
- CSS
- metadata

Then close it with:

html

```
</head>
```

Copy code

Step 4: `<meta charset="UTF-8">`

html

```
<meta charset="UTF-8">
```

Copy code

This tells the browser how to read text characters.

Why it matters:

- it helps display letters, punctuation, and symbols correctly
- avoids strange text errors

`UTF-8` is the standard and safest choice.

Step 5: `<title>Books Table</title>`

html

```
<title>Books Table</title>
```

Copy code

This sets the page title.

You usually see it:

- on the browser tab
- as the page name in bookmarks

It does **not** show as a big heading inside the page body.

Step 6: `<body>`

html

```html
<body>
```

Copy code

This is the main visible content of the webpage.

Everything inside `<body>` is what the viewer sees on the page.

Then close it with:

html

```html
</body>
```

Copy code

Step 7: `<h1>Books on Data Science</h1>`

html

```html
<h1>Books on Data Science</h1>
```

Copy code

This creates a large heading.

- `<h1>` means main heading
- it is usually the biggest title on the page

So on the webpage, this text appears as the page heading.

Step 8: `<table border="1">`

html

```html
<table border="1">
```

Copy code

This starts a table.

- `<table>` means you are creating a table
- `border="1"` adds visible borders around the cells

Without the border, the table may still exist, but it may not look obvious.

At the end, close it with:

```html
</table>
```

Copy code

Step 9: First `<tr>` = header row

```html
<tr>
  <th>Title</th>
  <th>Authors</th>
  <th>Publication Year</th>
  <th>Publisher</th>
  <th>ISBN</th>
</tr>
```

Copy code

What `<tr>` means

`<tr>` = **table row**

So this creates one row in the table.

What `<th>` means

`<th>` = **table header cell**

These are column names, such as:

- Title
- Authors
- Publication Year

- Publisher
- ISBN

Browsers usually display `<th>` text in **bold** and centered.

So this whole block is the **header row**.

Step 10: Second `<tr>` = first book row

html

```
<tr>
  <td>R for Data Science</td>
  <td>Hadley Wickham, Garrett Grolemund</td>
  <td>2017</td>
  <td>O'Reilly Media</td>
  <td>9781491910399</td>
</tr>
```

ChatGPT 5.4 Thinking

What `<td>` means

`<td>` = **table data cell**

Each `<td>` is one cell in the row.

So this row contains:

- title = R for Data Science
- authors = Hadley Wickham, Garrett Grolemund
- year = 2017
- publisher = O'Reilly Media
- ISBN = 9781491910399

Step 11: More `<tr>` rows for more books

html

```
<tr>
  <td>Data Science for Business</td>
  <td>Foster Provost, Tom Fawcett</td>
  <td>2013</td>
  <td>O'Reilly Media</td>
  <td>9781449361327</td>
</tr>
```

Copy code

This is the second book.

And:

```html
<tr>
  <td>Practical Statistics for Data Scientists</td>
  <td>Peter Bruce, Andrew Bruce, Peter Gedeck</td>
  <td>2020</td>
  <td>O'Reilly Media</td>
  <td>9781492072942</td>
</tr>
```

Copy code

This is the third book.

Each new book gets:

- one new `<tr>` row
- the same number of `<td>` cells as the header columns

Structure idea

Think of the table like this:

- `<table>` = whole table
- `<tr>` = one row
- `<th>` = header cell
- `<td>` = regular cell

So the logic is:

html Copy code

```html
<table>
  <tr>
    <th>column name</th>
    <th>column name</th>
  </tr>
  <tr>
    <td>row 1 data</td>
    <td>row 1 data</td>
  </tr>
  <tr>
    <td>row 2 data</td>
    <td>row 2 data</td>
  </tr>
</table>
```

Very simple meaning of the whole file

This HTML file does 3 things:

1. creates a webpage
2. shows a heading
3. displays a table of 3 books

What you would do for the assignment

For your assignment, the process is:

1. Pick a subject

Example:

- data science
- machine learning
- statistics
- algebra

2. Choose 3 books on that subject

Make sure:

- all are on the same topic
- at least one has multiple authors

3. Decide your columns

Required:

- title
- authors

Plus 2–3 extra attributes:

- year
- publisher
- ISBN

4. Put those column names in the header row

Use `<th>`

5. Add one row for each book

Use `<tr>` and `<td>`

6. Save the file as `books.html`

What the browser will show

When opened, the page will look roughly like:

Books on Data Science

Title	Authors	Publication Year	Publisher	ISBN
R for Data Science	Hadley Wickham, Garrett Grolemund	2017	O'Reilly Media	9781491910399
Data Science for Business	Foster Provost, Tom Fawcett	2013	O'Reilly Media	9781449361327

Title	Authors	Publication Year	Publisher	ISBN
Practical Statistics for Data Scientists	Peter Bruce, Andrew Bruce, Peter Gedeck	2020	O'Reilly Media	9781492072942

One more important detail

Because this assignment later uses **R** to read the HTML table, the table structure should stay simple and clean.

That means:

- one table
- one header row
- one book per row
- plain text inside cells

That makes it easier for R packages like `rvest` or `xml2` to read it.

Quick tag summary

Tag	Meaning
`<!DOCTYPE html>`	says this is an HTML5 file
`<html>`	wraps the whole page
`<head>`	page info
`<meta>`	text encoding
`<title>`	tab title
`<body>`	visible page content
`<h1>`	main heading

Tag	Meaning
`<table>`	starts a table
`<tr>`	table row
`<th>`	header cell
`<td>`	data cell

If you want, I can next explain the **JSON example step by step in the same way**.

ok

```
[
  {
    "title": "R for Data Science",
    "authors": "Hadley Wickham, Garrett Grolemund",
    "publication_year": 2017,
    "publisher": "O'Reilly Media",
    "isbn": "9781491910399"
  },
  {
    "title": "Data Science for Business",
    "authors": "Foster Provost, Tom Fawcett",
    "publication_year": 2013,
    "publisher": "O'Reilly Media",
    "isbn": "9781449361327"
  },
  {
    "title": "Practical Statistics for Data Scientists",
    "authors": "Peter Bruce, Andrew Bruce, Peter Gedeck",
    "publication_year": 2020,
    "publisher": "O'Reilly Media",
```

```
    "isbn": "9781492072942"
  }
]
```

Step by step

Copy code

1. Square brackets `[ ]`

```json
[
]
```

Copy code

These mean the whole file is a **list / array**.

Here, the list contains **3 books**.

So you can think of it as:

- book 1
- book 2
- book 3

2. Curly braces `{ }`

```json
{
}
```

Copy code

Each pair of curly braces is **one object**.

In this assignment:

- one object = one book

So this part:

```json
```

```
{
  "title": "R for Data Science",
  "authors": "Hadley Wickham, Garrett Grolemund",
  "publication_year": 2017,
  "publisher": "O'Reilly Media",
  "isbn": "9781491910399"
}
```

Copy code

means:

- this is one book record
- it has several fields

3. Keys and values

Example:

```json
"title": "R for Data Science"
```

Copy code

This has 2 parts:

- `"title"` = key / field name
- `"R for Data Science"` = value

So JSON is basically storing information as:

- field name → value

Another example:

```json
"publication_year": 2017
```

Copy code

means:

- the field is `publication_year`
- the value is `2017`

4. Double quotes `"`

In JSON:

- keys must use **double quotes**
- text values also use **double quotes**

Correct:

```json
"title": "R for Data Science"
```

Copy code

Not correct JSON:

```json
'title': 'R for Data Science'
```

Copy code

JSON is stricter than R.

5. Commas `,`

Commas separate items.

For example inside one book:

```json
{
  "title": "R for Data Science",
  "authors": "Hadley Wickham, Garrett Grolemund",
  "publication_year": 2017
}
```

Copy code

Each field is separated by a comma.

And between book objects:

```json
```

```
{
  ...
},
{
  ...
}
```

Copy code

the comma separates book 1 and book 2.

Important:

- use commas **between** items
- do **not** put an extra comma after the last item

First book explained

```json
{
  "title": "R for Data Science",
  "authors": "Hadley Wickham, Garrett Grolemund",
  "publication_year": 2017,
  "publisher": "O'Reilly Media",
  "isbn": "9781491910399"
}
```

Copy code

This means:

- title = R for Data Science
- authors = Hadley Wickham, Garrett Grolemund
- publication_year = 2017
- publisher = O'Reilly Media
- isbn = 9781491910399

This is the JSON version of one row from your HTML table.

Why the whole file starts with `[` and not just `{`

Because you have **multiple books**.

- `{}` = one book
- `[]` = a list of books

Since your assignment has 3 books, you usually wrap them in an array:

```json
[
  { book 1 },
  { book 2 },
  { book 3 }
]
```

Copy code

Match with HTML table

In HTML, your data looked like rows in a table.

In JSON, the same data is stored as objects.

HTML idea

- one row = one book

JSON idea

- one object = one book

So they contain the same information, just in different formats.

Data type detail

In JSON, values can be different types.

String

Text values use quotes:

```json
"title": "R for Data Science"
```

Copy code

Number

Numbers do not need quotes:

```json
"publication_year": 2017
```

Copy code

Also valid to keep everything as text

For simplicity, some people write:

```json
"publication_year": "2017"
```

Copy code

That is also okay, but then it is stored as text, not number.

For this assignment, either can work as long as your two data frames match properly after reading.

Structure idea

You can think of the file like this:

```json
[
  {
    "column1": "value",
    "column2": "value",
    "column3": "value"
  },
  {
    "column1": "value",
    "column2": "value",
    "column3": "value"
  }
]
```

Copy code

That means:

- the file contains a list
- each item in the list is one record
- each record has the same fields

Very simple meaning of the whole JSON file

This file does one thing:

- stores the same 3-book dataset in JSON format

What you would do for the assignment

1. Use the same 3 books from your HTML file

Do not change the data.

2. Use the same fields

For example:

- title
- authors
- publication_year
- publisher
- isbn

3. Create one object for each book

4. Put all 3 objects inside square brackets

5. Save as `books.json`

Quick symbol summary

Symbol	Meaning
`[ ]`	array / list
`{ }`	object / one record

Symbol	Meaning
:	separates key and value
,	separates items
"	required for JSON keys and text values

Compare HTML and JSON

Format	Structure
HTML	table with rows and columns
JSON	list of objects
Same thing	both store the same book data

Important for your assignment

Your JSON should stay simple, just like your HTML:

- one array
- one object per book
- same fields for each book
- no extra nesting unless needed

That makes it easier for R to read.

A more assignment-style mental model

You are making the same mini dataset twice:

In HTML

display it as a table

In JSON

store it as structured key-value records

Then in R, you read both and compare them.

Next step is usually the R part:

- read HTML
- read JSON
- compare the two data frames

I can explain the **R code step by step** next.

ok

Here is the **R part explained step by step**.

Example R code

```r
library(rvest)
library(jsonlite)

# Read HTML table
html_page <- read_html("books.html")
html_df <- html_table(html_page, fill = TRUE)[[1]]

# Read JSON file
json_df <- fromJSON("books.json")

# Compare the two data frames
identical(html_df, json_df)
```

Copy code

Step 1: load packages

```r
```

```
library(rvest)
library(jsonlite)
```

Copy code

What this does

- `rvest` is used to read HTML
- `jsonlite` is used to read JSON

So:

- HTML → use `rvest`
- JSON → use `jsonlite`

Step 2: read the HTML file

r

```
html_page <- read_html("books.html")
```

Copy code

What this does

This reads your HTML file into R.

- `"books.html"` is the file name
- `read_html()` opens and parses the HTML structure

Now `html_page` contains the HTML content, but not yet as a data frame.

Think of it as:

- first R opens the webpage/file
- then later you extract the table

Step 3: extract the table from the HTML

r

```
html_df <- html_table(html_page, fill = TRUE)[[1]]
```

Copy code

What this does

This finds the table inside the HTML and turns it into a data frame.

Break it apart

`html_table(html_page, fill = TRUE)`
This looks for tables in the HTML file.

If your page has one table, it will return a list containing that table.

`[[1]]`
This selects the first table.

Because your assignment usually has just one table, `[[1]]` gets that one.

Result

Now `html_df` is a data frame made from the HTML table.

Step 4: read the JSON file

r

```r
json_df <- fromJSON("books.json")
```

Copy code

What this does

This reads the JSON file and converts it into an R object.

Since your JSON is a list of book records, it will usually become a data frame automatically.

So now:

- `json_df` = data frame from JSON

Step 5: compare the two data frames

r

```
identical(html_df, json_df)
```

Copy code

What this does

This checks whether the two objects are exactly the same.

If they match perfectly, R returns:

```r
[1] TRUE
```

Copy code

If something is different, R returns:

```r
[1] FALSE
```

Copy code

What "identical" means

`identical()` is strict.

It checks things like:

- same column names
- same values
- same order
- same data types

So even if the data looks similar, `identical()` may return `FALSE` if:

- one column is character and the other is numeric
- the rows are in different order
- the column names are slightly different

Full logic of the code

The process is:

1. Read HTML

turn table into `html_df`

2. Read JSON

turn JSON into `json_df`

3. Compare

check whether `html_df` and `json_df` are exactly the same

Example of what the two data frames should look like

Both should end up looking something like this in R:

title	authors	publication_year	publisher	isbn
R for Data Science	Hadley Wickham, Garrett Grolemund	2017	O'Reilly Media	9781491910399
Data Science for Business	Foster Provost, Tom Fawcett	2013	O'Reilly Media	9781449361327
Practical Statistics for Data Scientists	Peter Bruce, Andrew Bruce, Peter Gedeck	2020	O'Reilly Media	9781492072942

If both files produce this same result, then `identical()` should return `TRUE`.

Why `fill = TRUE` is used

```r
html_table(html_page, fill = TRUE)
```

Copy code

This helps R handle tables safely if some rows are uneven.

For your assignment, your table should be clean anyway, but `fill = TRUE` is often added to avoid problems.

What each variable means

Variable	Meaning
`html_page`	raw HTML content
`html_df`	data frame created from HTML table
`json_df`	data frame created from JSON file

A slightly safer version

Sometimes HTML and JSON may read columns differently. For example:

- year from HTML may become character
- year from JSON may become numeric

Then `identical()` may return `FALSE`.

A safer version is:

r Copy code

```r
library(rvest)
library(jsonlite)

html_page <- read_html("books.html")
html_df <- html_table(html_page, fill = TRUE)[[1]]

json_df <- fromJSON("books.json")

html_df[] <- lapply(html_df, as.character)
json_df[] <- lapply(json_df, as.character)

identical(html_df, json_df)
```

Why this helps

This converts all columns in both data frames to character before comparing.

So if the values are the same but the types were different, they can still match.

Step by step for that extra part

r Copy code

```r
html_df[] <- lapply(html_df, as.character)
json_df[] <- lapply(json_df, as.character)
```

This means:

- take every column in `html_df`
- convert it to character
- do the same for `json_df`

Then compare again.

What to say in your assignment

You can write something simple like:

> I created the same book dataset in both HTML and JSON formats. Then I used R to read the HTML table and the JSON file into separate data frames. Finally, I compared the two data frames using `identical()` to confirm whether they contained the same data.

Very simple version of the workflow

File 1

`books.html`

contains a table

File 2

`books.json`

contains the same data in JSON

R code

- read HTML
- read JSON
- compare both

What you probably need to do next

Your actual assignment workflow is:

1. choose 3 books
2. make `books.html`
3. make `books.json`
4. run the R code
5. put all of it into a `.qmd` file

I can next give you a **full ready-to-use example** with:

- `books.html`
- `books.json`
- R code
- Quarto `.qmd` layout

one column read as double in html but chr in json.

Thought for a couple of seconds

That means the **same column was parsed with different data types**:

- in **HTML**: R read it as `double` / numeric
- in **JSON**: R read it as `character`

So `identical(html_df, json_df)` will return `FALSE` even if the visible values look the same.

Why this happens

R guesses column types when reading files.

Example:

- HTML table column might look like `2017, 2013, 2020` → R guesses **numeric**
- JSON might store them as `"2017", "2013", "2020"` → R reads them as **character**

Same value, different type.

How to check which column

Use:

```r
str(html_df)
str(json_df)
```

Copy code

or:

```r
sapply(html_df, class)
sapply(json_df, class)
```

Copy code

That will show which column is different.

Easiest fix

Convert the mismatched column in one data frame so both match.

For example, if `publication_year` is numeric in HTML and character in JSON:

```r
```

```
html_df$publication_year <- as.character(html_df$publication_year)
```

Copy code

or the other way:

r

```
json_df$publication_year <- as.numeric(json_df$publication_year)
```

Copy code

Then compare again:

r

```
identical(html_df, json_df)
```

Copy code

Safest assignment fix

Convert **all columns** in both data frames to character before comparing:

r

```
html_df[] <- lapply(html_df, as.character)
json_df[] <- lapply(json_df, as.character)

identical(html_df, json_df)
```

Copy code

This is usually the easiest for this assignment.

Full example

r

```
library(rvest)
library(jsonlite)

html_page <- read_html("books.html")
html_df <- html_table(html_page, fill = TRUE)[[1]]

json_df <- fromJSON("books.json")
```

```
html_df[] <- lapply(html_df, as.character)
json_df[] <- lapply(json_df, as.character)

identical(html_df, json_df)
```

Copy code

Better than `identical()`

You can also use:

```r
all.equal(html_df, json_df)
```

Copy code

This is less strict and can tell you what differs.

Best practical advice

For this assignment, use:

```r
html_df[] <- lapply(html_df, as.character)
json_df[] <- lapply(json_df, as.character)
```

Copy code

before comparing.

That avoids type mismatch problems.

If you want, I can give you a small code block that prints exactly **which column** is different.

Week7.qmd
File

It is done?

I'm checking the uploaded `.qmd` to see whether the required pieces are actually there and whether anything obvious is still missing.

I found the main issue already: the files are readable and the type-fix works, but your three books are not really on one narrow subject. That is the only requirement I'd treat as potentially risky.

Thought for 1m 11s

Almost.

What is already fine

Your **technical part is basically done**:

- you have a public `books.html` with a table and the fields `title`, `authors`, `year`, `publisher`, and `ISBN` GitHub +1
- you have a public `books.json` with the same 3 records and fields GitHub +1
- you found the type mismatch (`ISBN` as double in HTML vs character in JSON) and fixed it before running `identical()`

So from a **code/data** standpoint, this looks done.

What I would still fix before submitting

1. "Same topic" is a little risky

Your files are labeled **Mathematics**, and the three books are:

- *Abstract Algebra, 3rd Edition*
- *Statistical Inference*
- *Mathematics I* GitHub +1

That is broad **mathematics**, but not one tight subject.
A strict grader might say:

- algebra
- statistics
- general mathematics

are not the same topic.

Safer version:

- use **3 algebra books**, or
- use **3 statistics books**

2. Your written list does not fully match your actual data

In your `.qmd`, your third book description says **KESMIA Mounira**, but your actual HTML/JSON data uses **Augustus De Morgan** for *Mathematics I*.
That inconsistency should be corrected.

Bottom line

If you ask "can I submit it now?" — probably yes.
If you ask "is it fully clean and safest possible?" — not yet.

I would change only these two things:

- make all 3 books clearly one subject
- make the written description match the actual HTML/JSON exactly

Then it is clean.

Sources